NLP with Disaster

By Matthew Hawkes & Benny Fishman

DisasterNet Intelligence

Project Objectives

Goal:
Build a predictive model to determine whether or not a tweet is referring to a real disaster.

Success Metric:
Achieve a strong Kaggle submission score, and produce a clean, reproducible ML notebook.

Data Preprocessing

- Created two functions: clean_text and clean_text_raw.
- Both remove special characters and symbols.
- clean_text: tokenized text, removes stopwords and lemmatizes tokens.
- clean_text_raw: keeps stopwords to capture meaning in short tweets.

```python
def clean_text(text):
    text = re.sub(r'http\S+', '', text)
    text = re.sub(r'@[\w]+', '', text)
    text = re.sub(r'#\w+', '', text)
    text = re.sub(r'[^a-zA-Z\s]', '', text)
    text = text.lower()
```

```python
def clean_text_raw(text):
    text = re.sub(r'http\S+', '', text)
    text = re.sub(r'@[\w]+', '', text)
    text = re.sub(r'#(\w+)', r'\1', text)
    text = re.sub(r'[^a-zA-Z\s]', '', text)
    text = text.lower().strip()
    return text
```

```python
    # Remove stopwords + very short tokens
    tokens = [word for word in tokens if word not in stop_words and len(word) > 2]
    tokens = [lemmatizer.lemmatize(word) for word in tokens]

    return tokens
```

EDA: Dataset Structure

- Determined the number of rows/tweets in the test dataframe.
- Found the distribution of target class in dataframe.
- Checked our columns, which include new names (tokens, cleaned_text).
- Found the most common words in all of the tweets.
- Knowing class balance affects how we evaluate models.

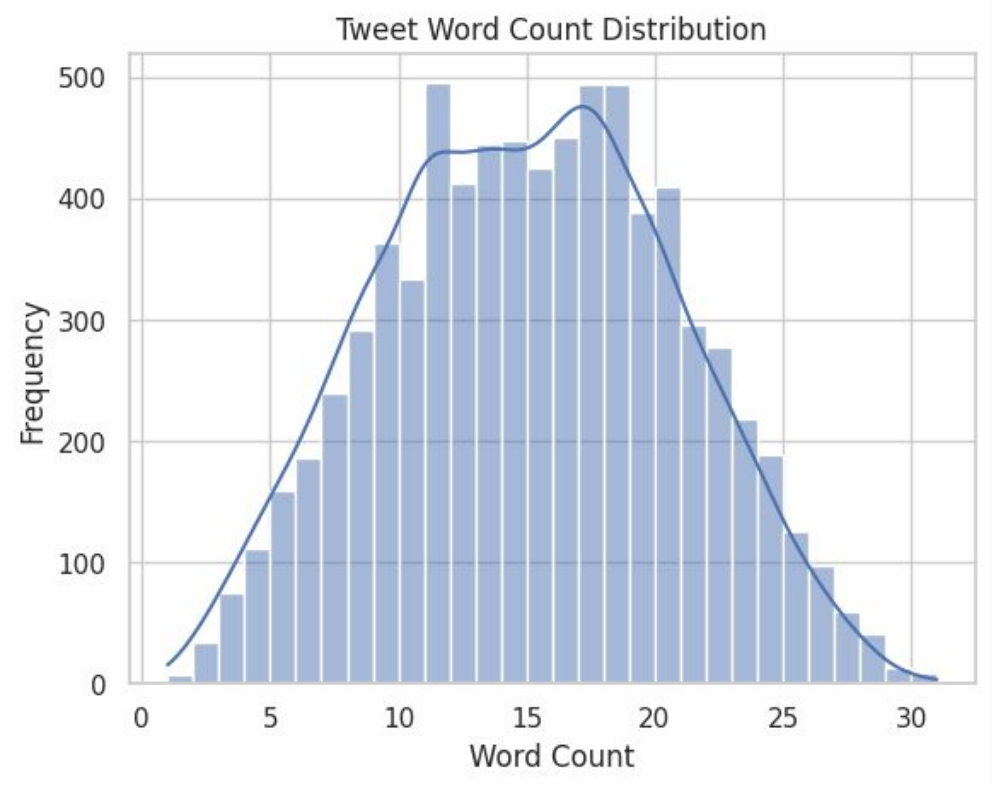
```
Normalized number of tweets with target=0: 0.5710897351429701
Normalized Number of tweets with target=1: 0.42891026485702993
```

```
Number of Tweets: 7613
Column Names: id, keyword, location, text, target, tokens, cleaned_text
```

```
[('fire', 350),
 ('like', 346),
 ('amp', 300),
 ('get', 255),
 ('new', 226),
 ('via', 220),
 ('dont', 208),
 ('one', 205),
 ('people', 197),
 ('time', 174),
 ('video', 170),
 ('emergency', 153),
 ('year', 150),
 ('body', 150),
 ('disaster', 148),
 ('day', 141),
 ('building', 140),
 ('police', 140),
 ('home', 139),
 ('would', 131)]
```

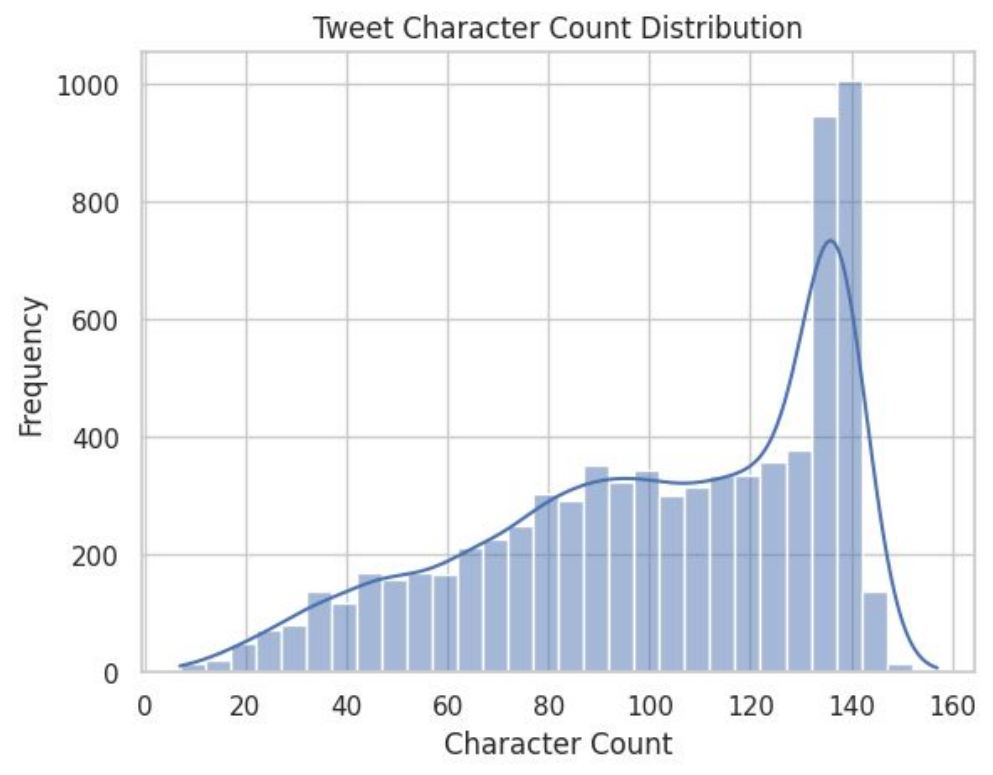
EDA: Word Count Distribution by Class

- Majority of tweets fall within a 10-20 word range.
- Models need to capture meaningful word patterns not just length.
- Short length indicates that every word and pattern carries importance.



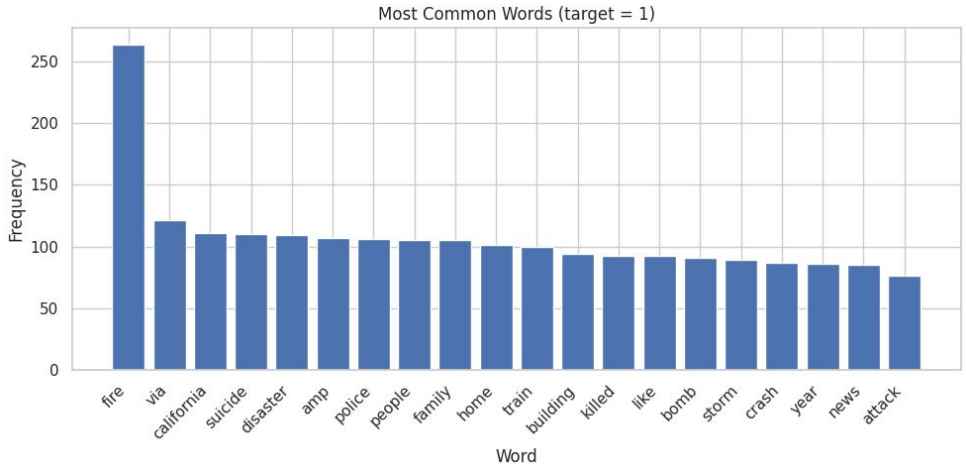
EDA: Char Count Distribution by Class

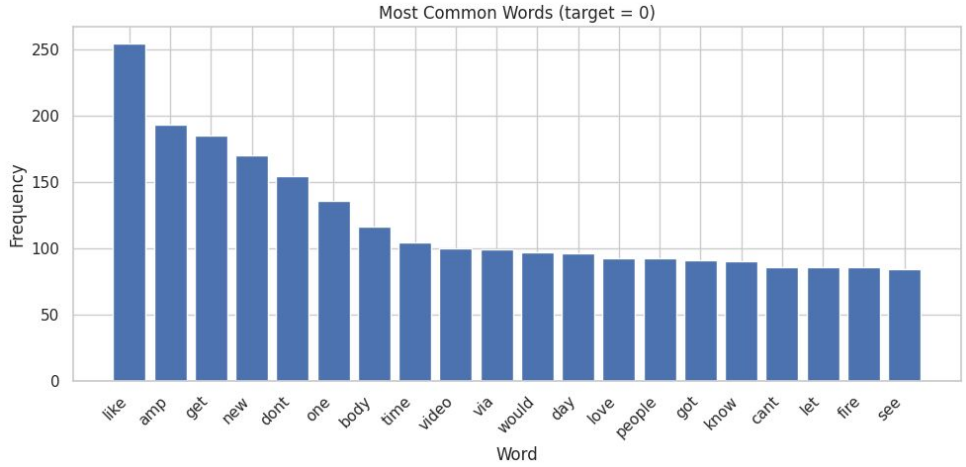
- Most tweets fall within a very narrow character range.
- Classification is mostly driven by context and patterns rather than message size.
- Supports the use of char n-grams which capture small variations in text.



EDA: Comparison

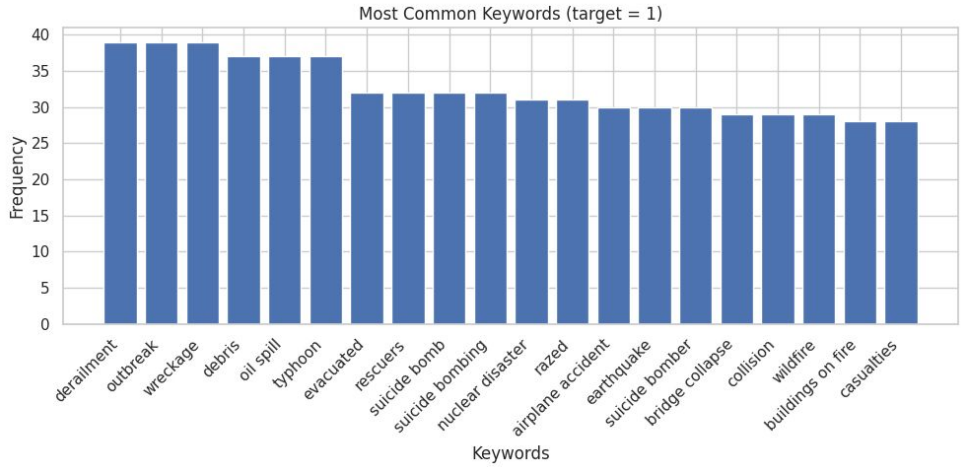
- Compared common words between real and fake disaster tweets.
- Certain words appear much more frequently in real disaster tweets, while others appear in fake ones.
- This confirmed that text contains meaningful predictive signal.

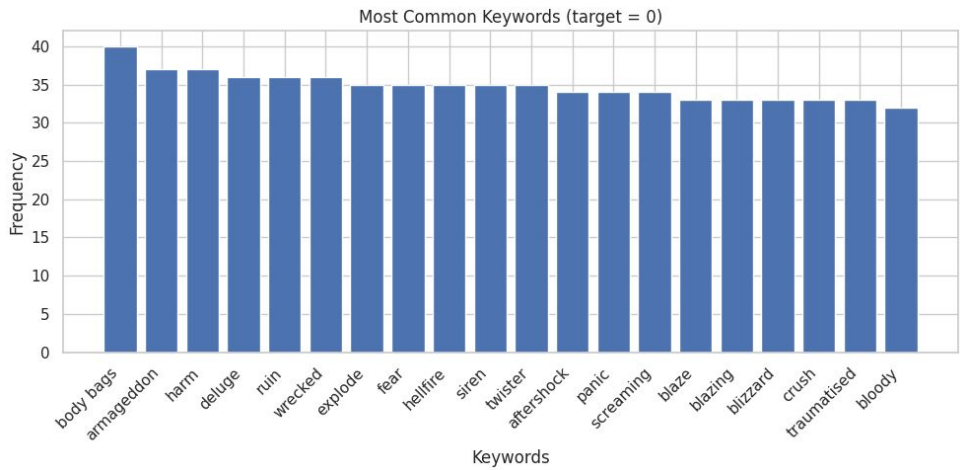




EDA: Comparison

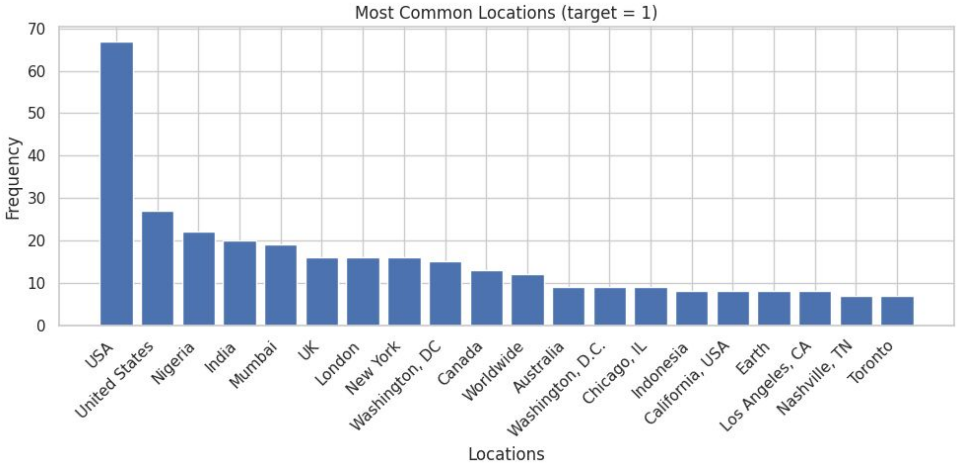
- Compared common keywords between real and fake disaster tweets.
- High-signal column with lots of predictive value.
- Influenced our text vectorization choices.

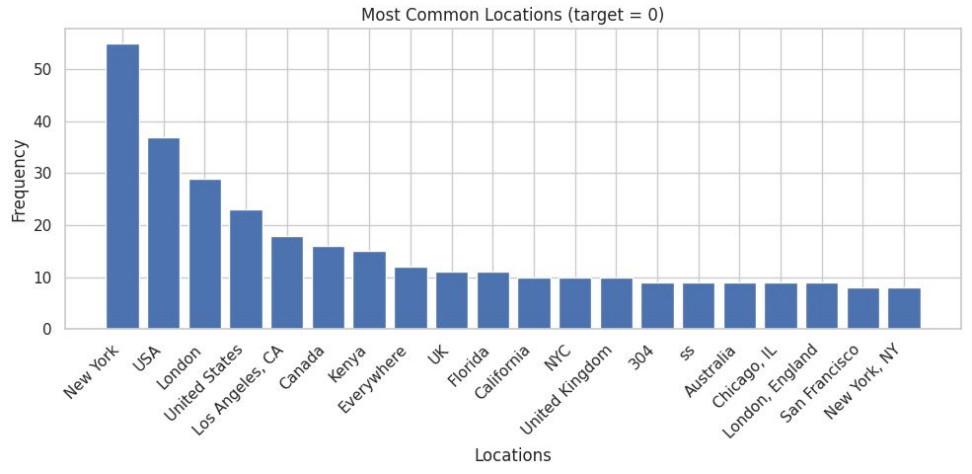




EDA: Comparison

- Compared common locations between real and fake disaster tweets.
- Non-disaster tweets frequently had no location or fake locations.
- Location data was messy and inconsistent.
- It would require complex cleaning or encoding to be useful.





Text Vectorization

- Stacked 4 TF-IDF vectorizers into one sparse feature matrix.
- Word unigrams+bigrams on lemmatized combined text.
- Character n-grams (2-5) on raw combined text to capture shorter insight.
- Keyword unigrams to capture the high-signal column.
- Stacked the 4 vectorizers into two variables: X_train, X_test.

```
X_word_train  = word_tfidf.fit_transform(train_data['combined_text'])
X_char_a_train = char_tfidf_a.fit_transform(train_data['combined_raw'])
X_char_b_train = char_tfidf_b.fit_transform(train_data['combined_raw'])
X_kw_train    = kw_tfidf.fit_transform(train_data['cleaned_keyword'].fil

X_word_test   = word_tfidf.transform(test_data['combined_text'])
X_char_a_test  = char_tfidf_a.transform(test_data['combined_raw'])
X_char_b_test  = char_tfidf_b.transform(test_data['combined_raw'])
X_kw_test     = kw_tfidf.transform(test_data['cleaned_keyword'].fillna(
```

```
X_train = hstack([X_word_train, X_char_a_train, X_char_b_train, X_kw_train])
X_test  = hstack([X_word_test,  X_char_a_test,  X_char_b_test,  X_kw_test])
```

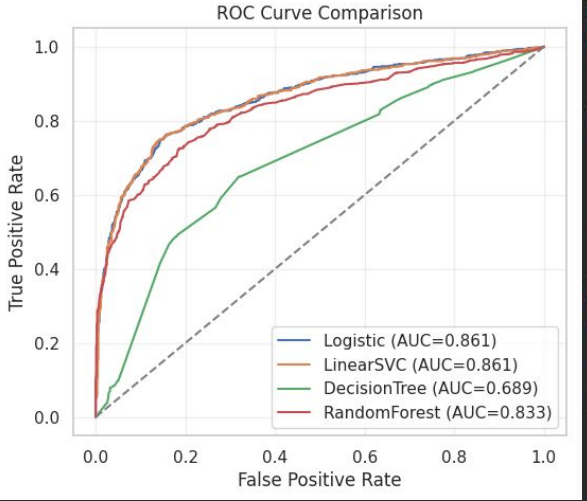
Determining the Best Model

- Split data for baseline model comparison using 80/20 proportion.
- Created a pipeline to compare four baseline models:
    1. Logistic Regression
    2. LinearSVC
    3. Decision Tree
    4. Random Forest

```
pipelines = {
    'Logistic': Pipeline([
        ('model', LogisticRegression(max_iter=1000, C=1.0, class_weight='balanced'))
    ]),
    'LinearSVC': Pipeline([
        ('model', LinearSVC(class_weight='balanced', random_state=42, max_iter=3000, C=0.1))
    ]),
    'DecisionTree': Pipeline([
        ('model', DecisionTreeClassifier(
            max_depth=8,
            min_samples_leaf=3,
            class_weight='balanced',
            random_state=42
        ))
    ]),
    'RandomForest': Pipeline([
        ('model', RandomForestClassifier(random_state=42, n_estimators=100))
    ]),
}
```

Baseline Results

- AUC: LinearSVC equal Logistic Regression meaning both best distinguish between classes.
- Accuracy: LinearSVC correctly classifies the most tweets.
- F1 Score: LinearSVC has the highest score meaning it provides the best tradeoff between finding positives and avoiding false alarms.
- Each score indicates that LinearSVC is the most effective model.



	Model	Accuracy	Precision	Recall	F1-score
1	LinearSVC	0.805665	0.774092	0.763240	0.768627
0	Logistic	0.801054	0.763158	0.767913	0.765528
3	RandomForest	0.778656	0.798828	0.637072	0.708839
2	DecisionTree	0.679842	0.663866	0.492212	0.565295

```
Logistic
[[723 153]
 [149 493]]
              precision    recall  f1-score   support

Fake Disaster      0.83      0.83      0.83       876
Real Disaster      0.76      0.77      0.77       642

    accuracy                           0.80      1518
   macro avg       0.80      0.80      0.80      1518
weighted avg       0.80      0.80      0.80      1518


LinearSVC
[[733 143]
 [152 490]]
              precision    recall  f1-score   support

Fake Disaster      0.83      0.84      0.83       876
Real Disaster      0.77      0.76      0.77       642

    accuracy                           0.81      1518
   macro avg       0.80      0.80      0.80      1518
weighted avg       0.81      0.81      0.81      1518


DecisionTree
[[716 160]
 [326 316]]
              precision    recall  f1-score   support

Fake Disaster      0.69      0.82      0.75       876
Real Disaster      0.66      0.49      0.57       642

    accuracy                           0.68      1518
   macro avg       0.68      0.65      0.66      1518
weighted avg       0.68      0.68      0.67      1518


RandomForest
[[773 103]
 [233 409]]
              precision    recall  f1-score   support

Fake Disaster      0.77      0.88      0.82       876
Real Disaster      0.80      0.64      0.71       642

    accuracy                           0.78      1518
   macro avg       0.78      0.76      0.77      1518
weighted avg       0.78      0.78      0.77      1518
```

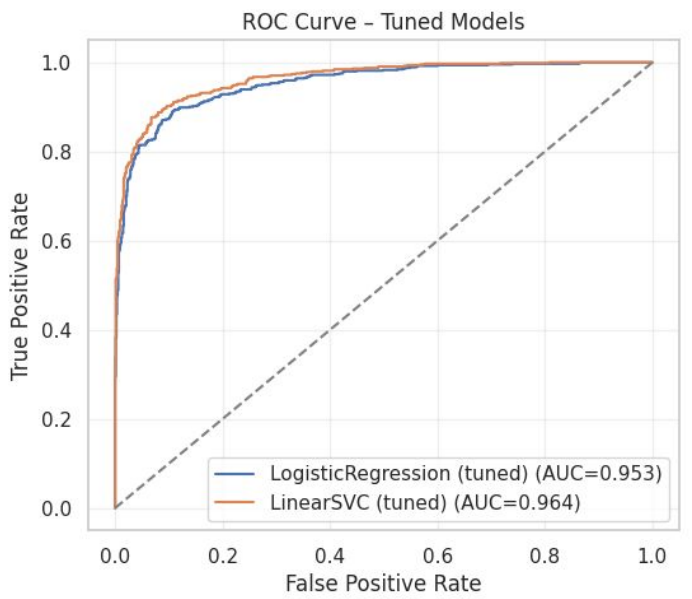
Hyperparameter Tuning

- Tuned both a Logistic Regression and a LinearSVC model, strong baseline models.
- Determined LinearSVC is the best model for this problem.

```
# Hyperparameter dictionary for LR search range
param_distributions_lr = {
    'model__C':            loguniform(2e-1, 5.0),
    'model__class_weight': [None, 'balanced'],
}

grid_lr = RandomizedSearchCV(
    lr_pipe,
    param_distributions_lr,
    n_iter=40,
    scoring='f1',
    cv=cv,
    n_jobs=-1,
    verbose=1,
    random_state=RANDOM_STATE
)
```

```
# Stratified CV keeps class balance across every fold
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=RANDOM_STATE)
```



	Model	Accuracy	Precision	Recall	F1-score
1	LinearSVC (tuned)	0.906456	0.899361	0.876947	0.888013
0	LogisticRegression (tuned)	0.892622	0.884430	0.858255	0.871146

Prediction

- Using LinearSVC, Predicted off the test dataset and created submission file.
- Submitted file to Kaggle for scoring.
- Original Score: 0.79987 →0.80202

Danyal Husain	0.79987	1	16d
Matthew Hawkes	0.79987	6	18m

Your Best Entry!
Your most recent submission scored 0.78577, which is not an improvement of your previous score. Keep trying!

Николай Лагодюк	0.79957	3	2mo

Leyuza Köksöken	0.80202	1	15d
Matthew Hawkes	0.80202	15	2h

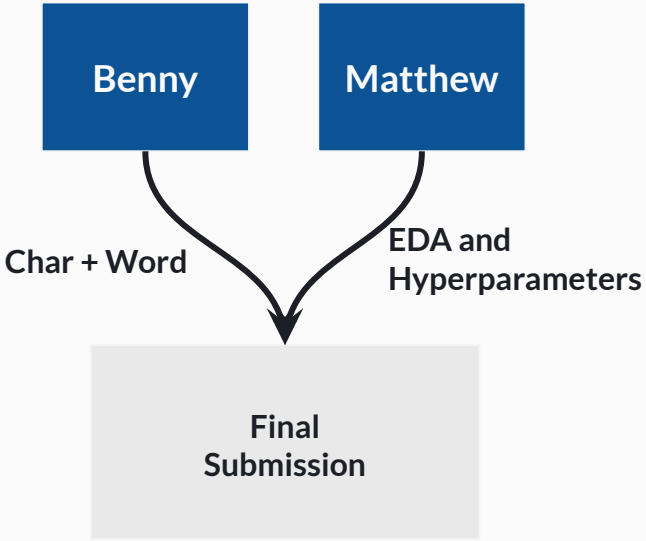
Your Best Entry!
Your most recent submission scored 0.80202, which is an improvement over your previous score of 0.79987. Great job!

Tweet this

AnnNTNU	0.80171	3	2mo

Advantages of Collaboration

- Allowed us to identify areas of weakness and strength within our projects.
- Benny's code's biggest contributions:
  - Splitting the text into both character and word vectorizers
  - Allowed the model to pick up subtle signal a lot better
- Matthew's code's biggest contributions:
  - Extensive EDA
  - Effective hyperparameter tuning
- Achieved a higher final score.



Limitations & Next Steps

- Ambiguous tweets can cause the model to be confused.
- Lack of semantic understanding made the model not understand relationships between words.
- Limited data count to produce a robust model.
- Use deep learning models to capture even more context and understanding.
- Engineer features regarding the tweet's performance.
- Use location as a feature by keeping track of disaster-prone areas/regions.

Reflection

- Feature engineering (char + word TF-IDF) was key for greater model performance.
- Collaboration allowed us to combine our strengths for an improved Kaggle score (0.79987 → 0.80202).
- EDA helped guide modeling decisions and revealed meaningful differences between real and fake disaster tweets.
- Data preprocessing matters more than complex models (deep learning).

Thank You